Sistemas Operativos de Tiempo Real (2)

Técnicas Digitales III

RTOS: Características

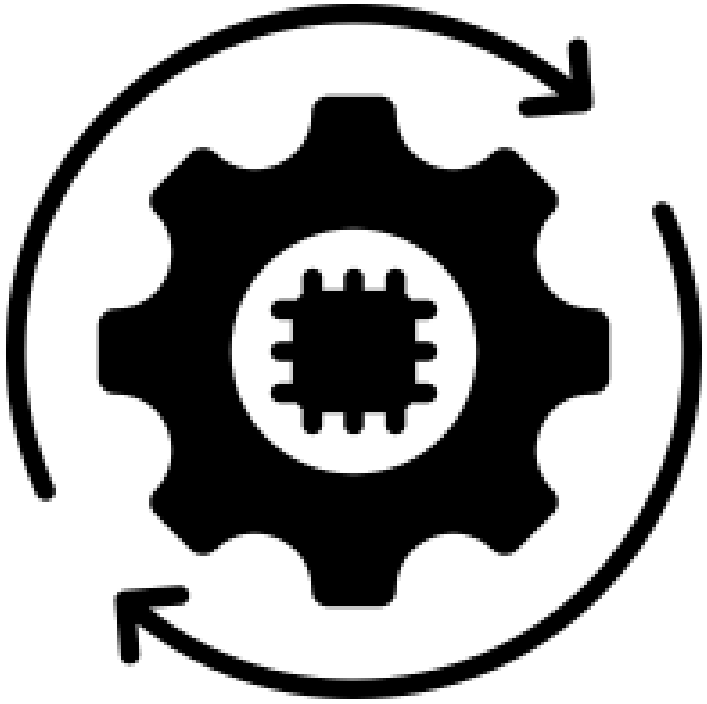
- Está diseñado específicamente para garantizar el tiempo de respuesta requerido por estas aplicaciones críticas
- Estructura: se asemeja más a una biblioteca de funciones enlazada con la aplicación principal

RTOS: Características

- La mayoría de los SOTR para sistemas empotrados no se protegen frente a errores de las aplicaciones para simplificar su diseño y el hardware necesario.
- Componentes:
  - Mecanismos para compartir datos.
  - Soporte de ISR.
  - Funciones para gestión de tiempo.

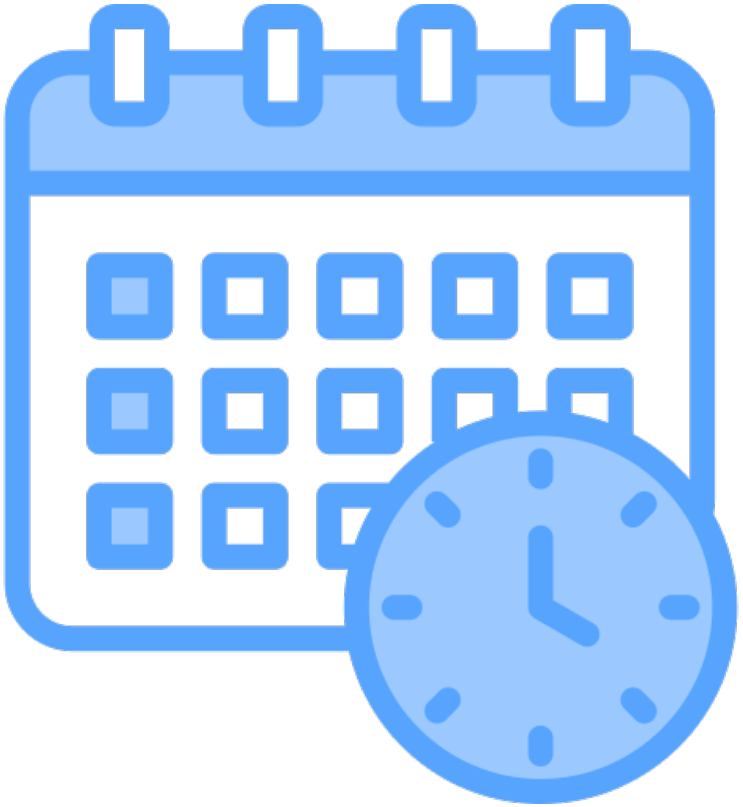
RTOS: Características

- Tareas:
  - El bloque básico de un programa basado en SOTR es la tarea.
  - Una tarea es una función de C que contiene un bucle infinito y nunca termina.
  - Las tareas se inicializan mediante llamadas al sistema operativo, especificando su prioridad, memoria necesaria y punto de entrada.
  - Los SOTR pueden ejecutar un número arbitrario de tareas, limitado por la memoria.
  - El número de prioridades puede ser configurable y limitado, ya que cada nivel consume memoria



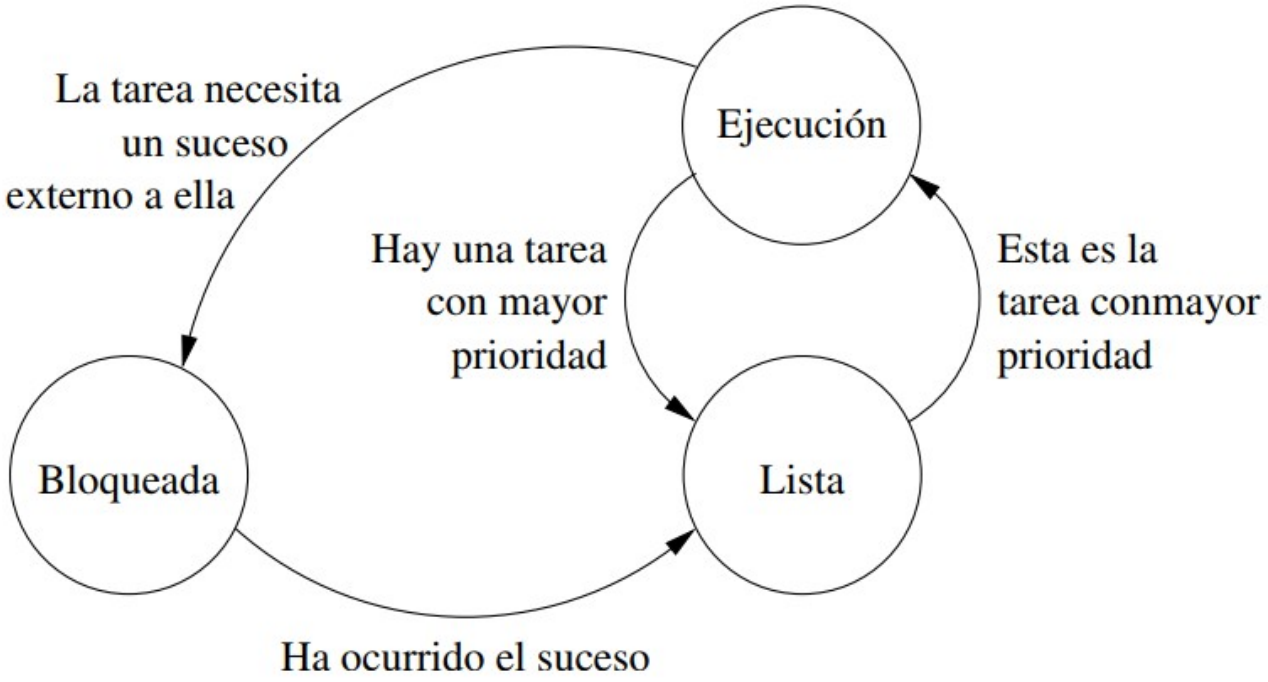
RTOS: Características

- Planificador:
  - Es un componente principal que controla el estado de cada tarea (Ejecución, Lista, Bloqueada) y decide cuándo una tarea pasa al estado de ejecución.
  - La regla general es que la tarea con mayor prioridad que está lista (*ready*) para ejecutarse es la que se ejecuta.



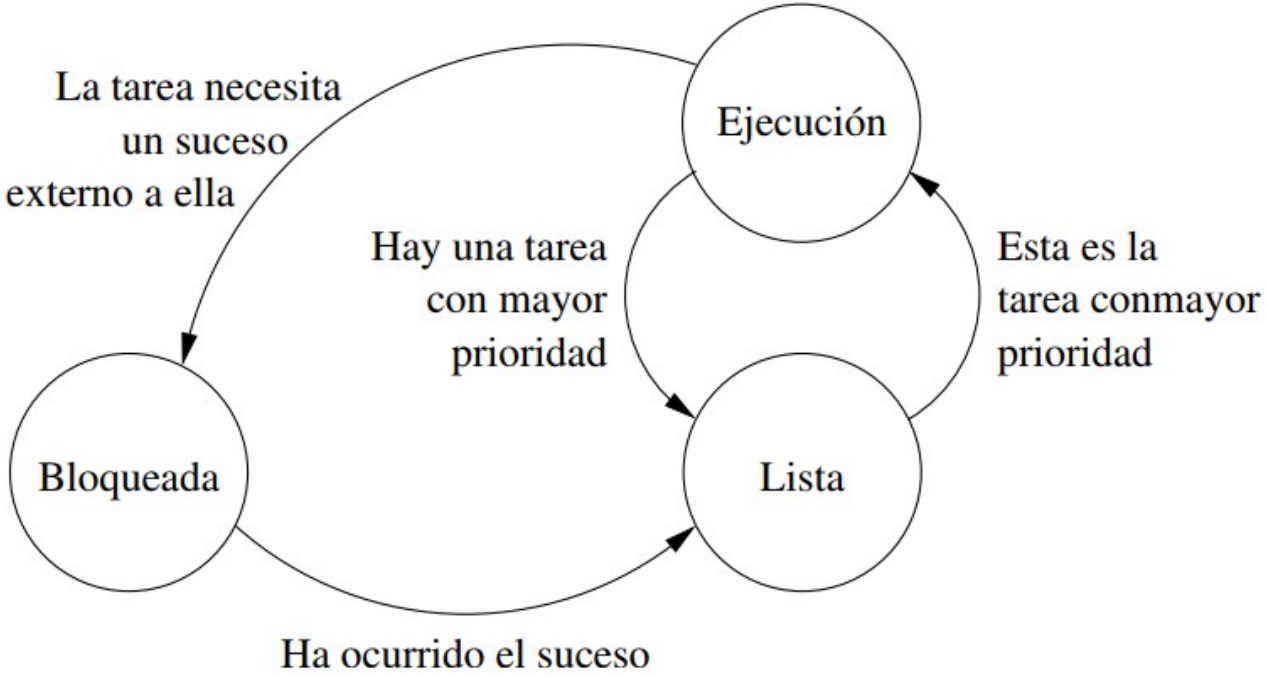
RTOS: Características

- Una tarea se bloquea mediante una llamada al sistema operativo, generalmente cuando espera un evento (como datos de una cola) o un período de tiempo.



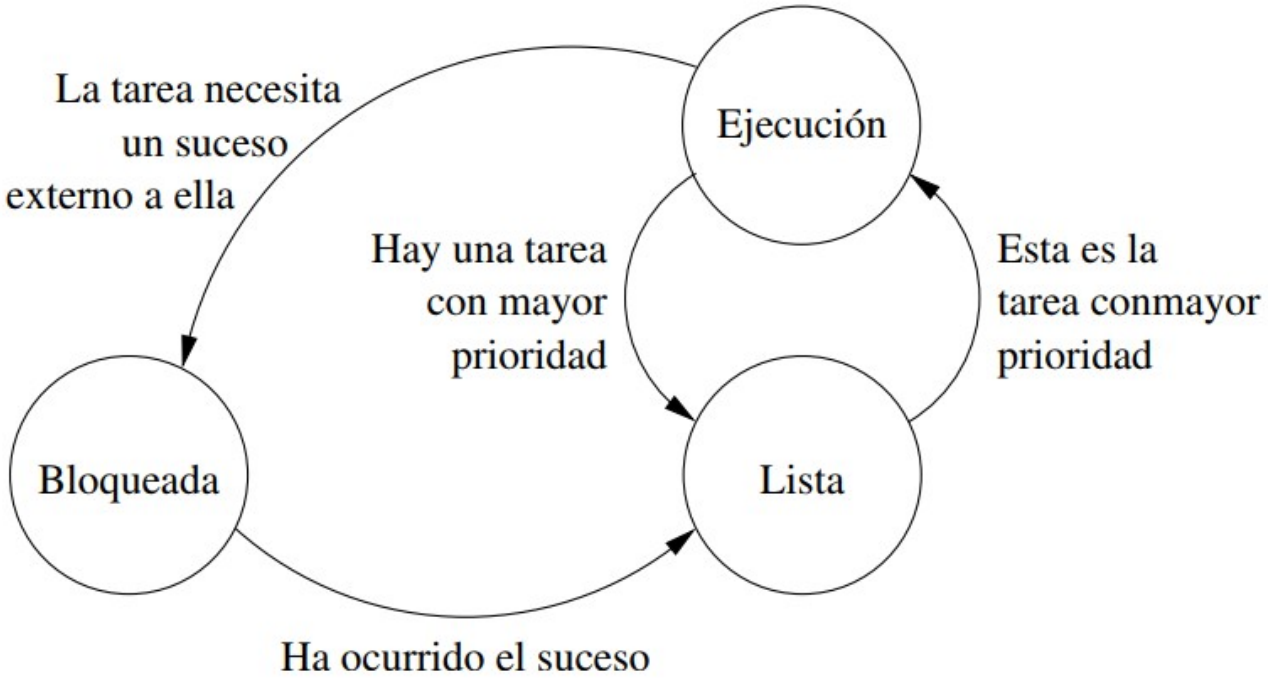
RTOS: Características

- Una tarea bloqueada pasa al estado lista cuando el evento esperado ocurre o el tiempo de espera termina.



RTOS: Características

- Si todas las tareas están bloqueadas, se ejecuta una tarea inactiva (*idle task*) .

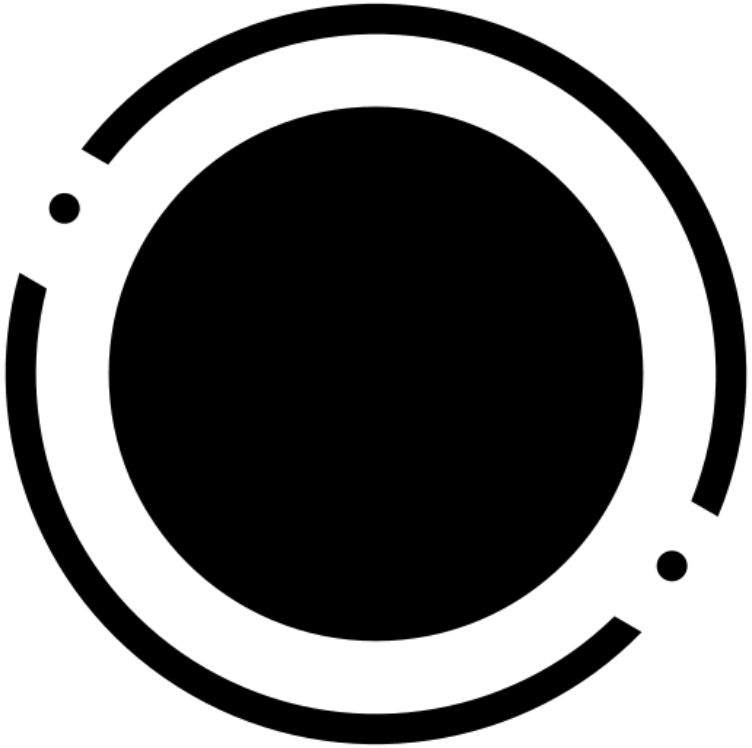


Tipos de planificador

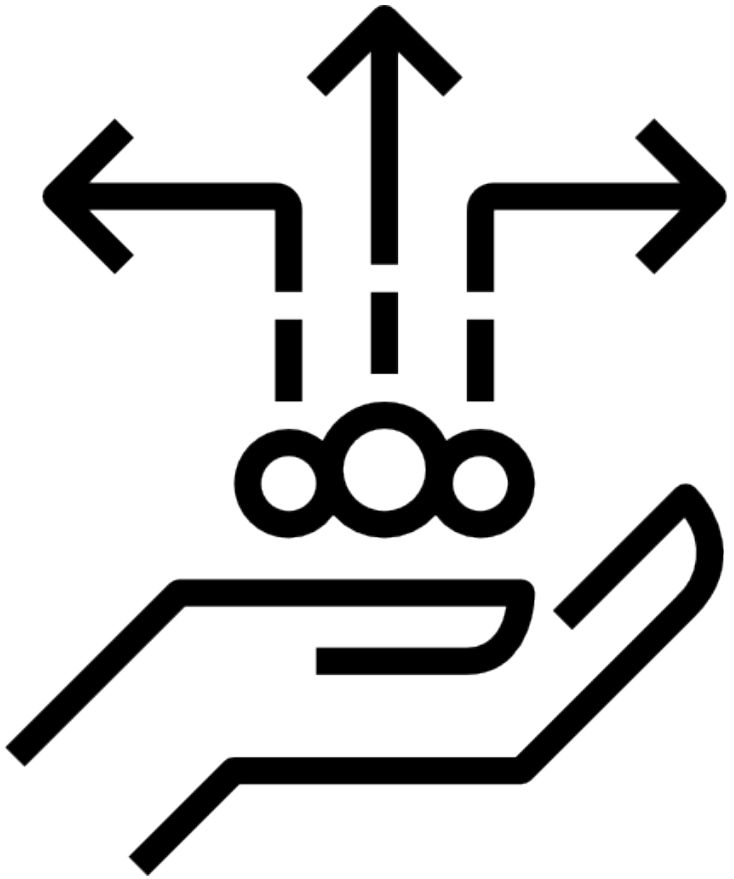
- **Cooperativo**:
  - Las tareas deben ceder explícitamente el control del procesador.
  - Es más simple y eficiente en recursos limitados.
  - La latencia depende de cuán a menudo las tareas ceden la CPU.

Tipos de planificador

- **Expropiativo** (*Preemptive*):
  - El planificador se ejecuta periódicamente mediante una interrupción de temporizador.
  - Puede interrumpir una tarea de menor prioridad para ejecutar una de mayor prioridad que está en estado *lista*.
  - Mejora la latencia de las tareas de mayor prioridad. El intervalo de tiempo (*time slice*) es configurable, impactando la latencia y la sobrecarga del sistema.



Recursos compartidos



Características FreeRTOS

- Operación cooperativa o expropiativa

Segmentación de tiempo opcional (time-slicing)

Asignación muy flexible de prioridades de tareas

Mecanismos de notificación de tareas flexibles, rápidos y livianos

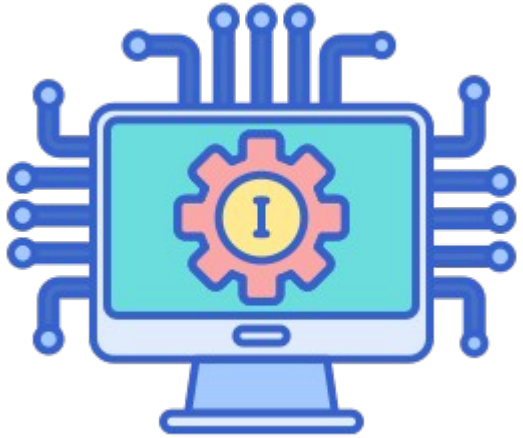
Colas (Queues)

Semáforos binarios

Semáforos contadores

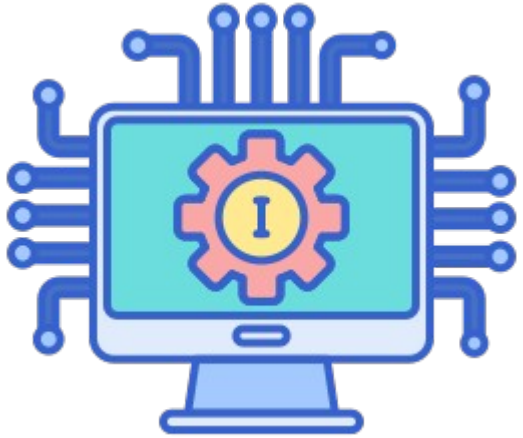
Mutexes (exclusión mutua)

Mutexes recursivos



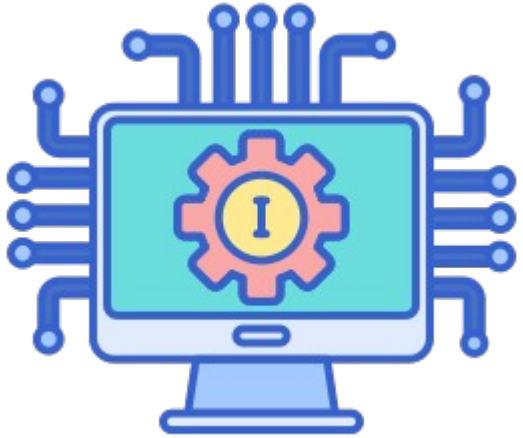
Características FreeRTOS

- Temporizadores por software Grupos de eventos
- Buffers de flujo (stream buffers)
- Buffers de mensajes (message buffers)
- Co-rutinas (en desuso / obsoletas)
- Funciones hook para el tick del sistema
- Funciones hook para la tarea en estado inactivo (idle)
- Verificación de desbordamiento de pila (stack overflow)



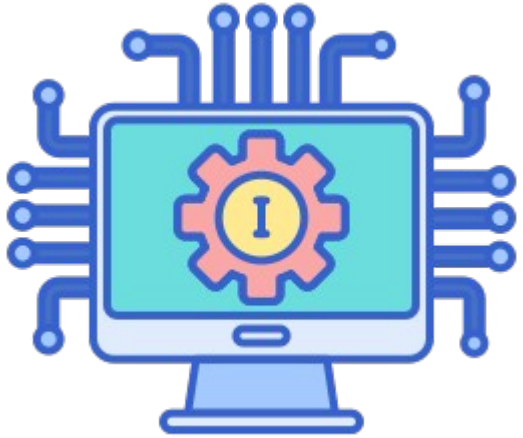
Características FreeRTOS

- Macros de trazado (trace macros)
- Recolección de estadísticas de tiempo de ejecución de tareas
- Licenciamiento y soporte comercial opcional
- Modelo completo de anidamiento de interrupciones (para algunas arquitecturas)
- Capacidad sin ticks (tick-less) para aplicaciones de ultra bajo consumo (en algunas arquitecturas)



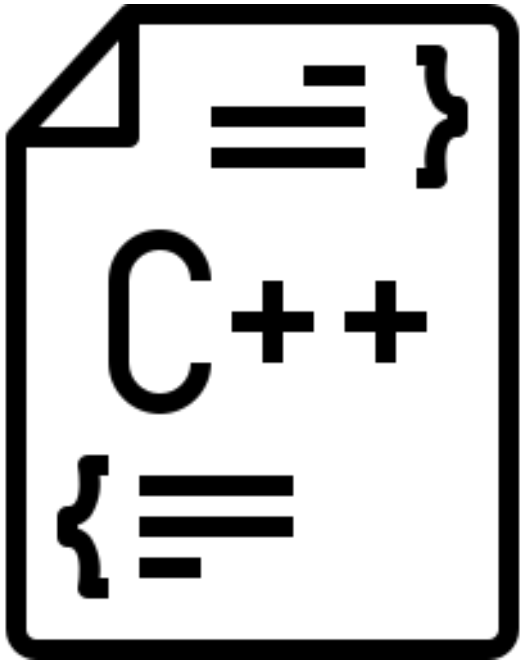
Características FreeRTOS

- Soporte para la Unidad de Protección de Memoria (MPU) para aislar tareas y aumentar la seguridad de la aplicación (en algunas arquitecturas)
- Pila de interrupciones gestionada por software cuando es apropiado (esto puede ayudar a ahorrar RAM)
- Capacidad de crear objetos del RTOS usando memoria asignada estática o dinámicamente



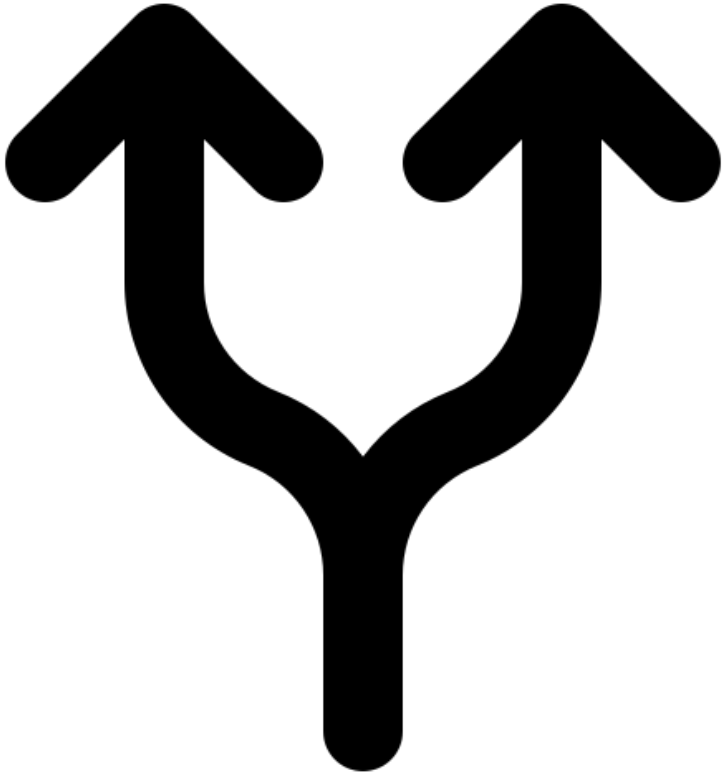
Tareas FreeRTOS

- **Bloque Básico de Programa:** Una tarea es el elemento fundamental de un programa que se ejecuta sobre un sistema operativo en tiempo real.
- **Función C Infinita:** En esencia, una tarea no es más que una función en lenguaje C que nunca termina, es decir, contiene un bucle infinito donde realiza sus acciones. Aunque puede llamar a otras funciones, la tarea en sí misma no debe retornar



Tareas FreeRTOS

- **Ejecución Paralela:** Los SOTR, como FreeRTOS, permiten la ejecución de un número arbitrario de tareas "en paralelo", limitado únicamente por la memoria disponible en el sistema



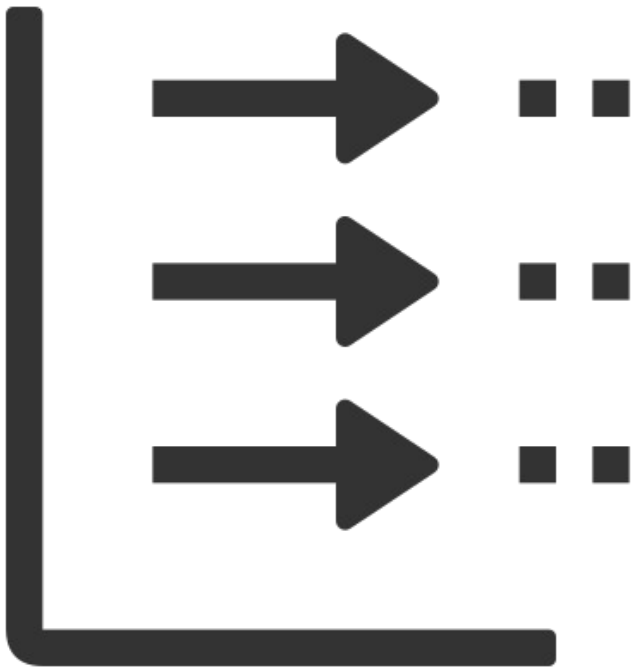
Creación de tareas

- **Llamada al Sistema Operativo:** Una tarea se inicializa mediante una llamada al SOTR. En FreeRTOS, esto se realiza comúnmente con la función xTaskCreate.

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        unsigned short usStackDepth,
                        void *pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t *pxCreatedTask );
```

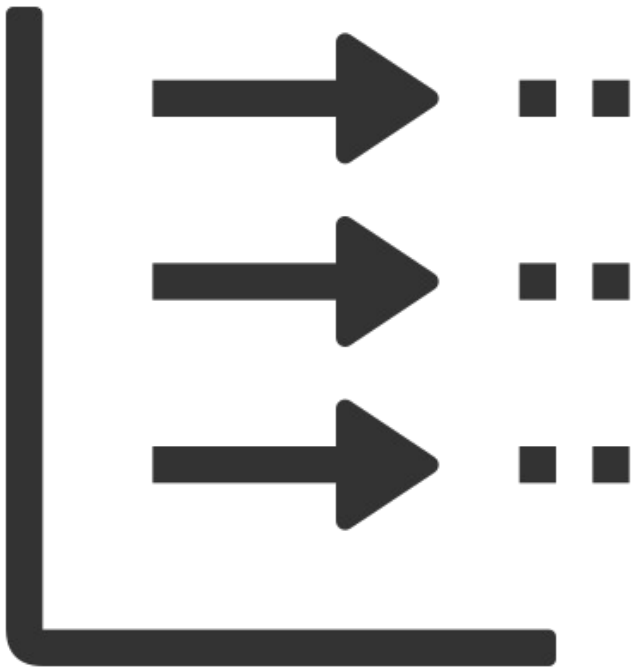
Creación de tareas

- Parámetros:
  - **Punto de Entrada (pvTaskCode):** La dirección de la función C que implementa la tarea.
  - **Prioridad (uxPriority):** La prioridad de la tarea. En FreeRTOS, el número de prioridades es configurable, y se recomienda mantenerlo lo más bajo posible ya que cada nivel consume memoria para estructuras de control. Por defecto, FreeRTOS tiene un máximo de 5 prioridades configurables, pero puede ser hasta 2^32 - 1 en versiones para ColdFire.



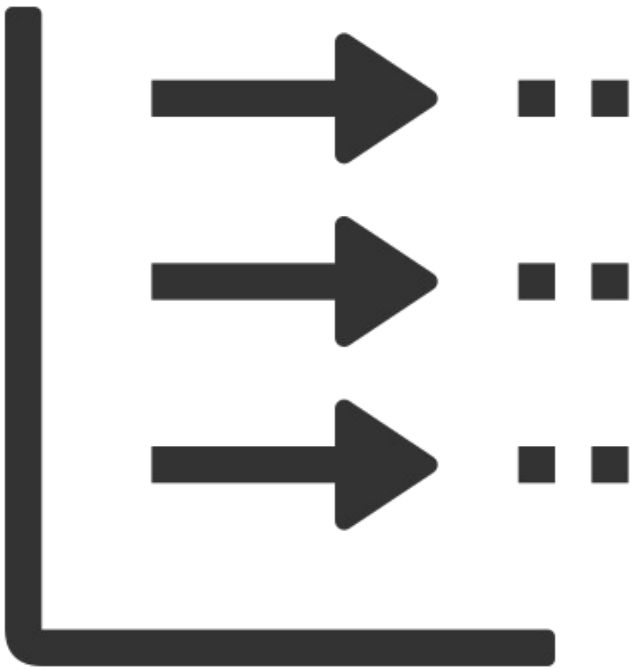
Creación de tareas

- Parámetros:
  - **Memoria (usStackDepth):** La cantidad de memoria de pila que necesita la tarea. Esta pila es creada automáticamente por el sistema operativo. El tamaño se especifica en palabras (4 Bytes). Para funciones sin I/O, 100 palabras pueden ser suficientes, pero si se usan funciones de entrada/salida (como sprintf), se pueden necesitar hasta 1000 palabras.
  - **Nombre Descriptivo (pcName):** Un nombre simbólico, útil para depuración, con una longitud máxima de 10 caracteres en FreeRTOS.



Creación de tareas

- Parámetros:
  - **Parámetros Iniciales (pvParameters):** Un puntero a los datos iniciales de la tarea; si no se necesitan, se puede dejar como NULL.
  - **Manejador (pvCreatedTask):** Un puntero para devolver una estructura de control de la tarea, útil si se desea destruir la tarea durante el funcionamiento del sistema.



Estados de tareas

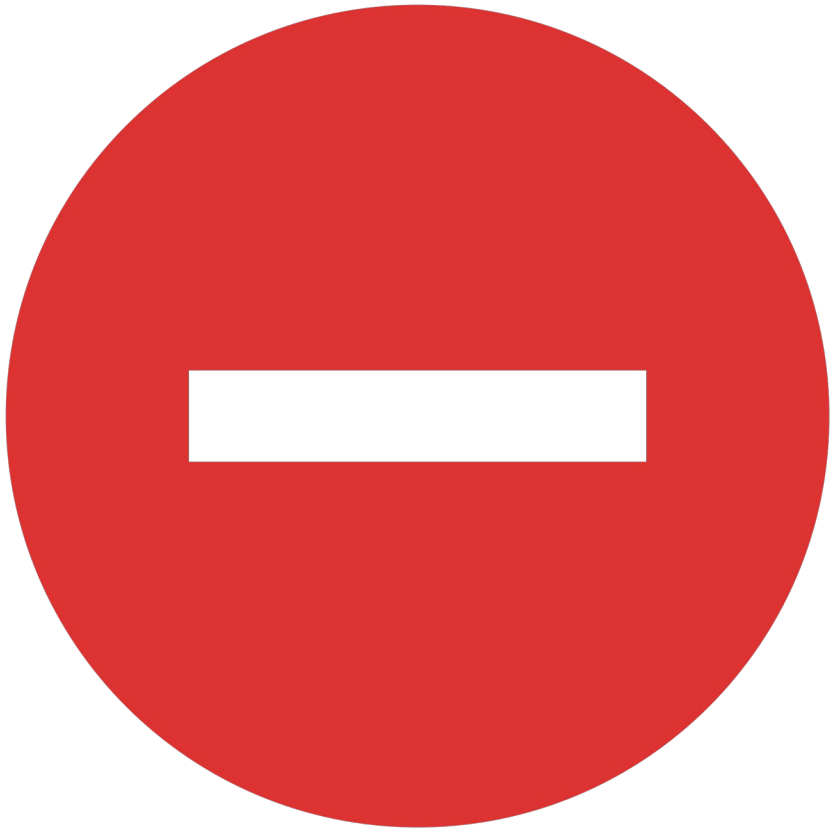
- **Ejecución (Running):** La tarea está siendo ejecutada por el microprocesador. Solo puede haber una tarea en este estado a la vez.

Estados de tareas

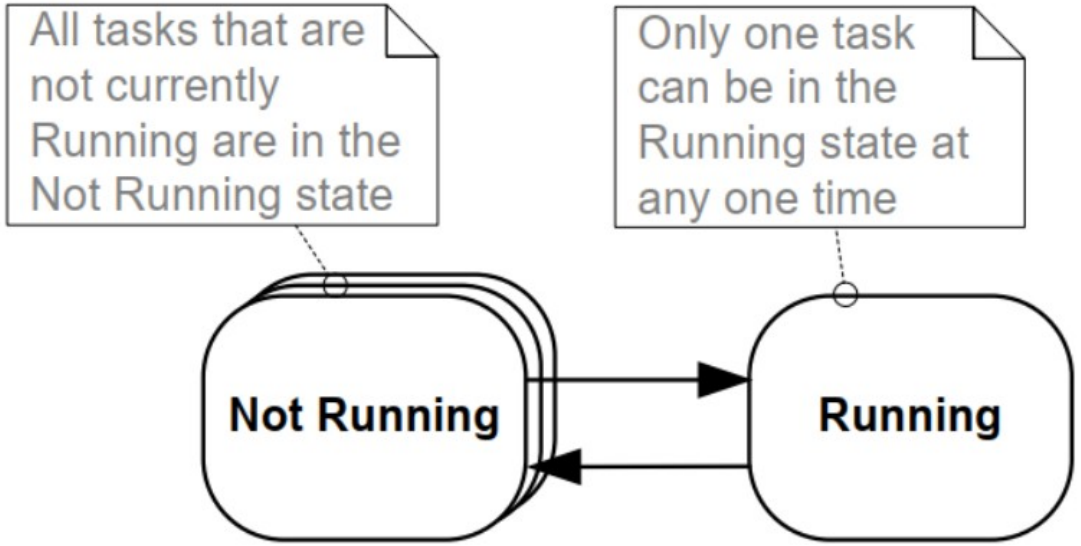
- **Lista (Ready):** La tarea tiene trabajo que hacer y está esperando a que el procesador esté disponible para ejecutarse. Puede haber varias tareas en este estado.

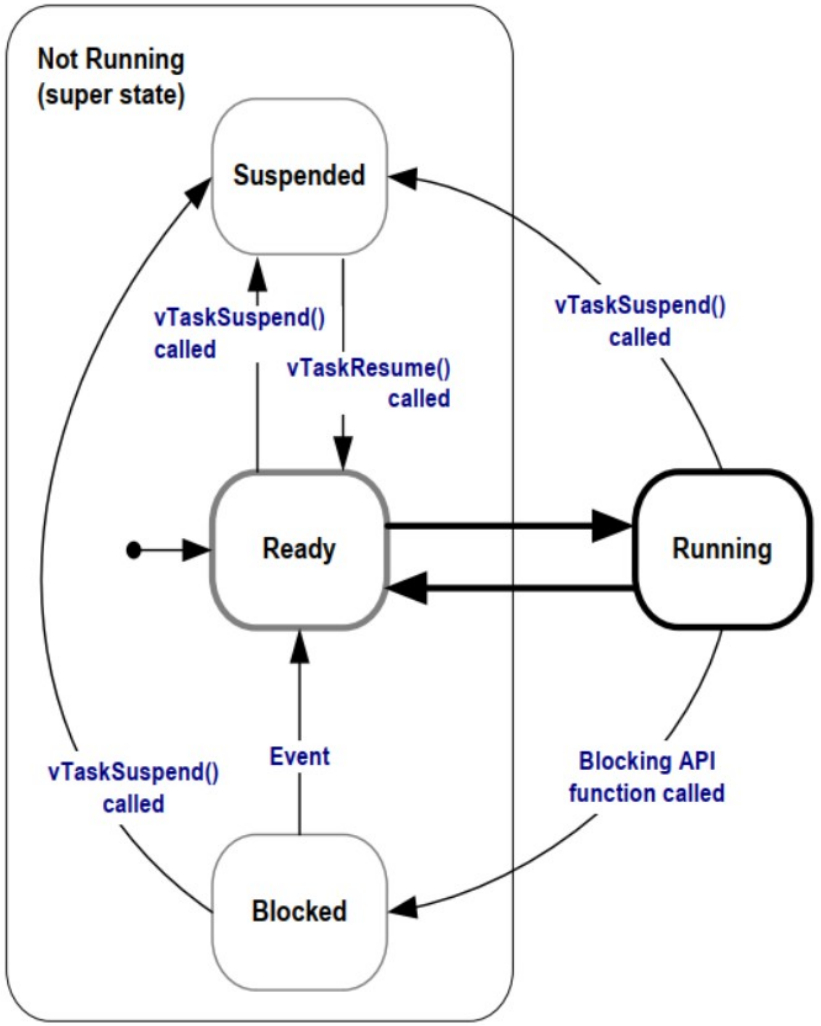
Estados de tareas

- **Bloqueada (Blocked):** La tarea no tiene nada que hacer en este momento y está esperando que ocurra algún suceso externo (por ejemplo, datos en una cola, liberación de un semáforo o que transcurra un tiempo específico). Una tarea solo puede bloquearse cuando está en ejecución.



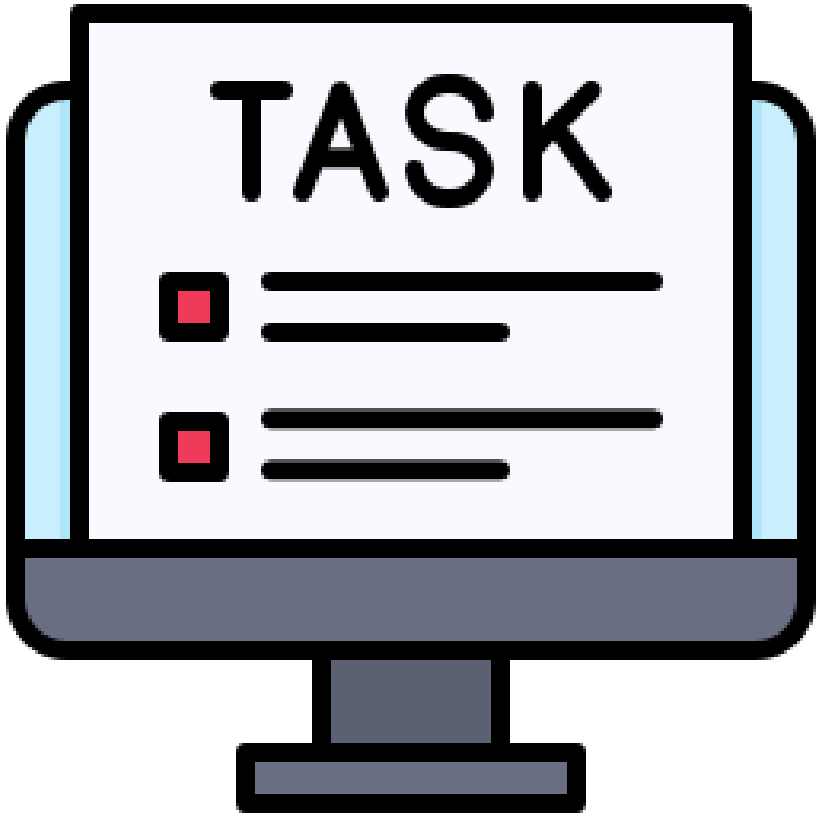
Estados de tareas





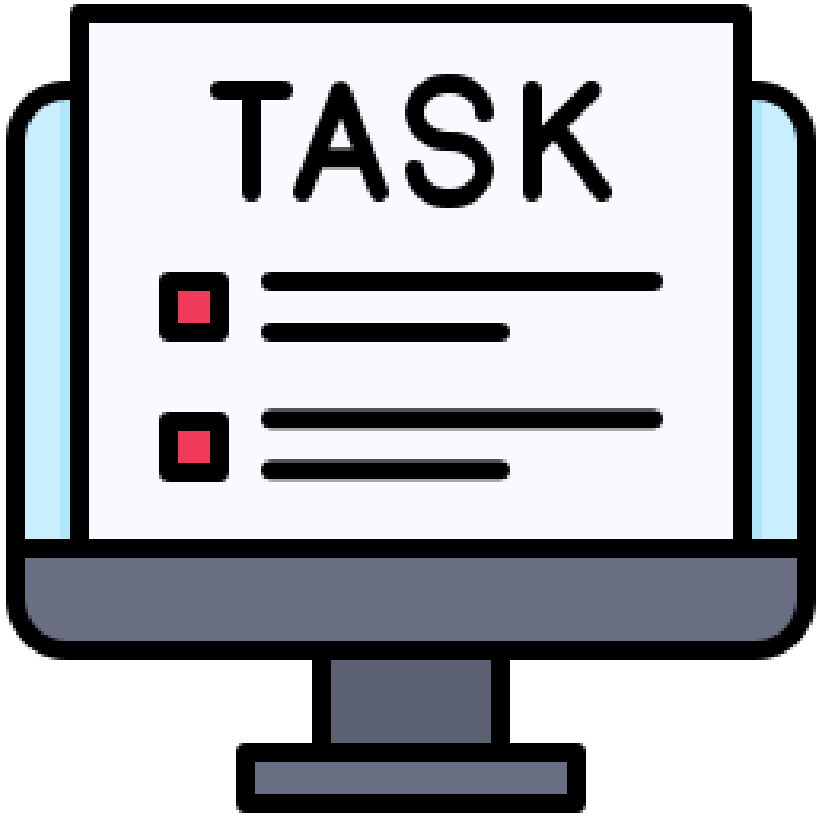
Planificador

- **Control del Estado:** El planificador (scheduler) es la parte del SOTR que controla el estado de cada tarea y decide cuándo una tarea pasa al estado de ejecución.



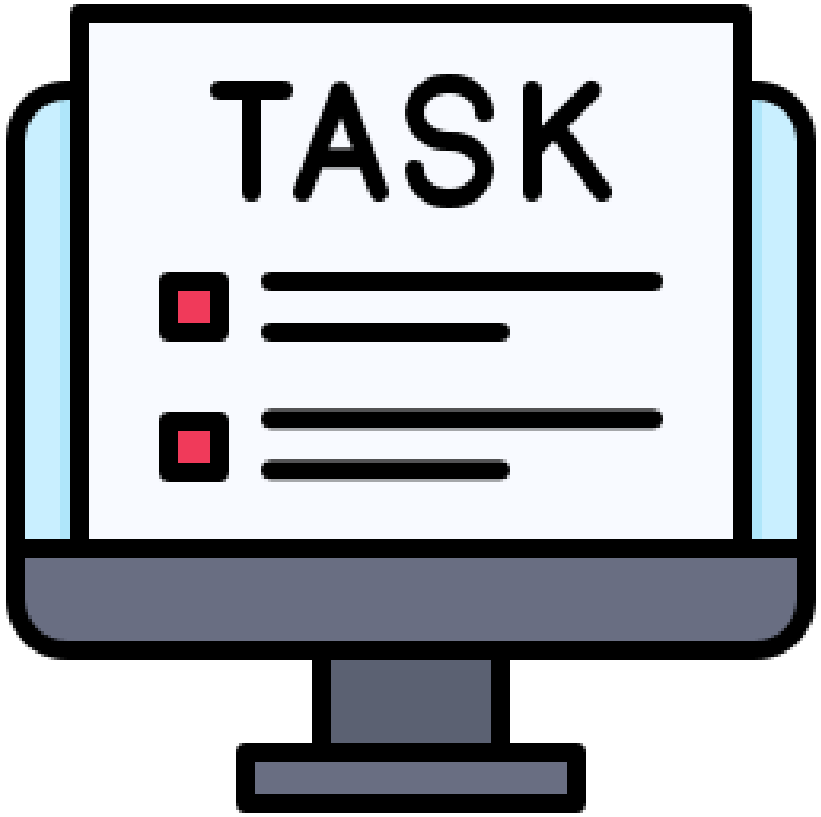
Planificador

- **Prioridad Determinante:** La regla principal es que la tarea con mayor prioridad que esté en estado "Lista" es la que se ejecuta. A diferencia de los sistemas operativos de propósito general que distribuyen tiempo de CPU entre tareas de baja prioridad, los planificadores de SOTR son más "brutos": si una tarea de alta prioridad monopoliza el procesador, las de menor prioridad simplemente



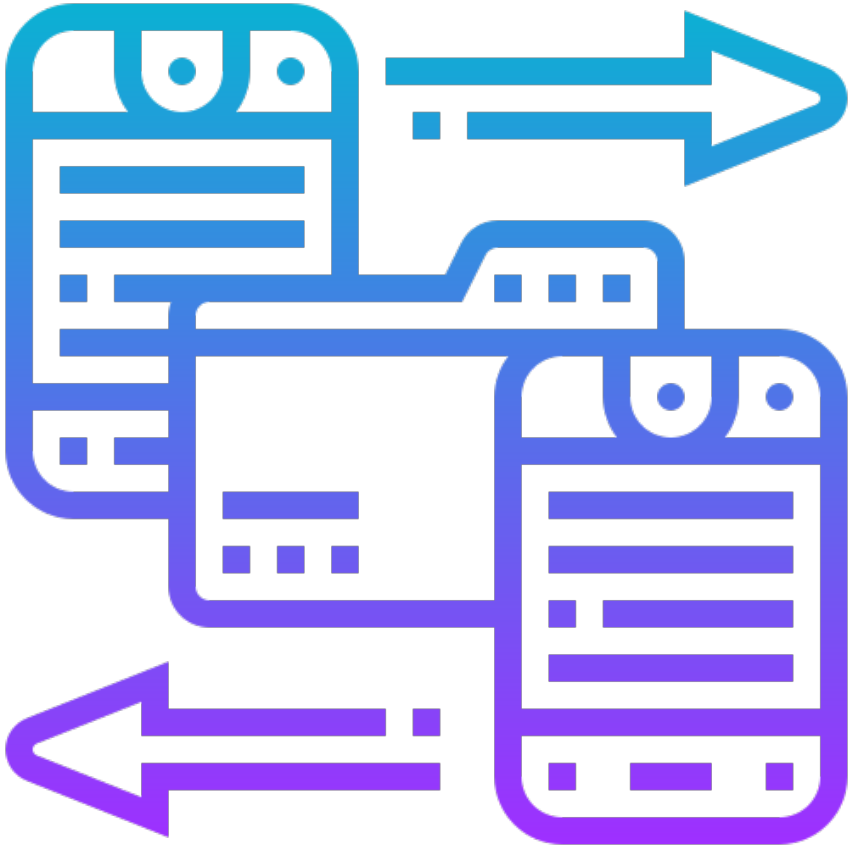
Planificador

- **Cambio de Contexto:** Para que el cambio entre tareas sea transparente al programador, el SOTR se encarga de guardar el contexto (registros del microprocesador, contador de programa, pila) de la tarea que deja de ejecutarse y cargar el contexto de la nueva tarea que toma el control.



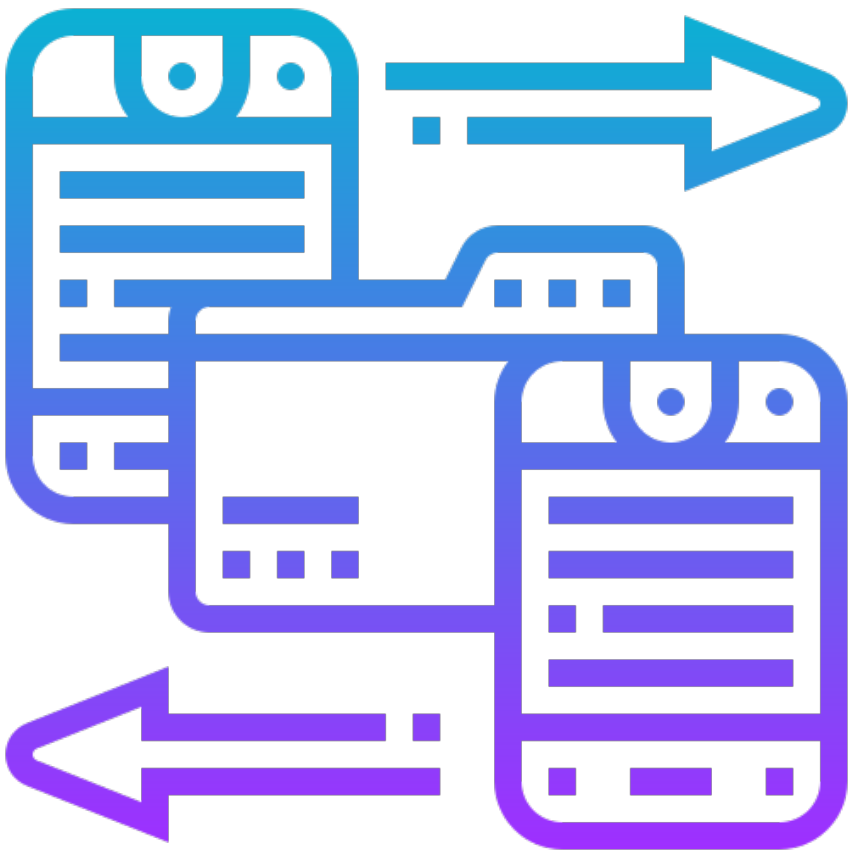
Datos compartidos

- **Variables Globales:** Los datos compartidos (variables globales) no forman parte del contexto individual de una tarea y, si son accedidos por múltiples tareas en paralelo, requieren precauciones para evitar problemas de coherencia de datos.



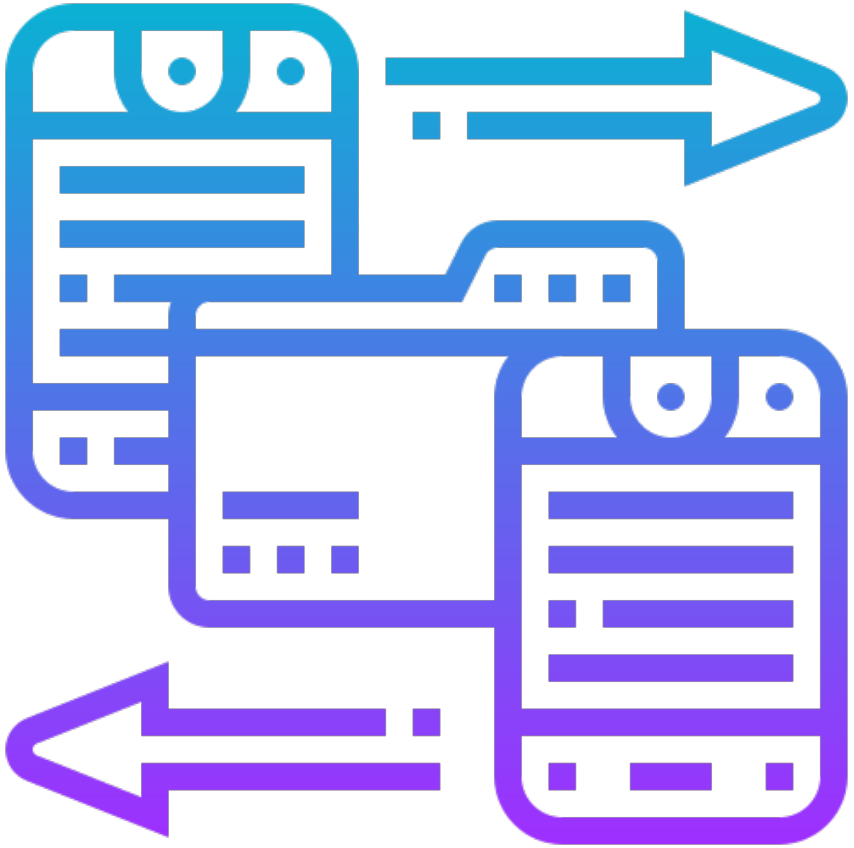
Datos compartidos

- **Funciones Reentrantes:** Una función es reentrante si puede ser llamada desde dos tareas distintas sin causar problemas. Para ser reentrante, una función debe:
  - Usar variables de forma atómica, a menos que estén almacenadas en la pila de la tarea que la llama.
  - Solo llamar a funciones reentrantes.
  - Usar el hardware de forma atómica.



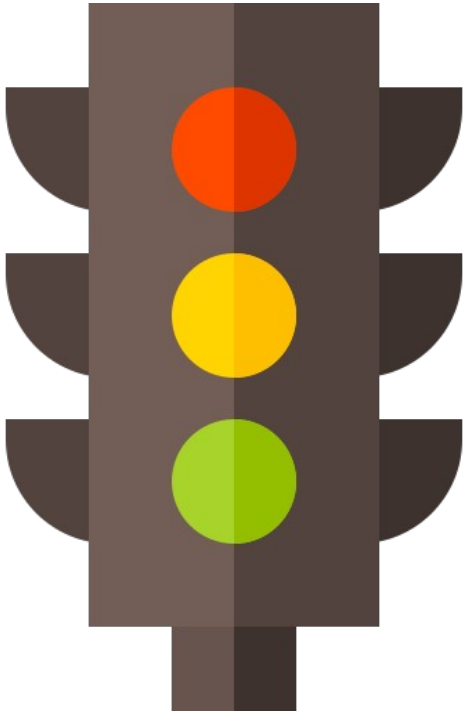
Datos compartidos

- **Problemas de Coherencia:** Una interrupción o cambio de contexto puede llevar a la pérdida de una actualización si no se maneja atómicamente.



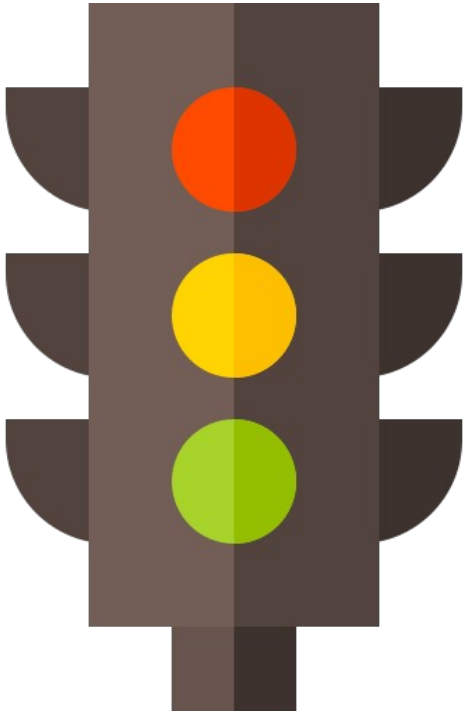
Sincronización y comunicación

- Semáforos:
  - **Protección de Recursos Compartidos:** Su uso principal es proteger secciones críticas de código, asegurando que solo una tarea acceda a un recurso compartido a la vez (exclusión mutua).
  - En FreeRTOS, se usan xSemaphoreTake() para adquirir y xSemaphoreGive() para liberar un semáforo. Es fundamental inicializar el semáforo con vSemaphoreCreateBinary



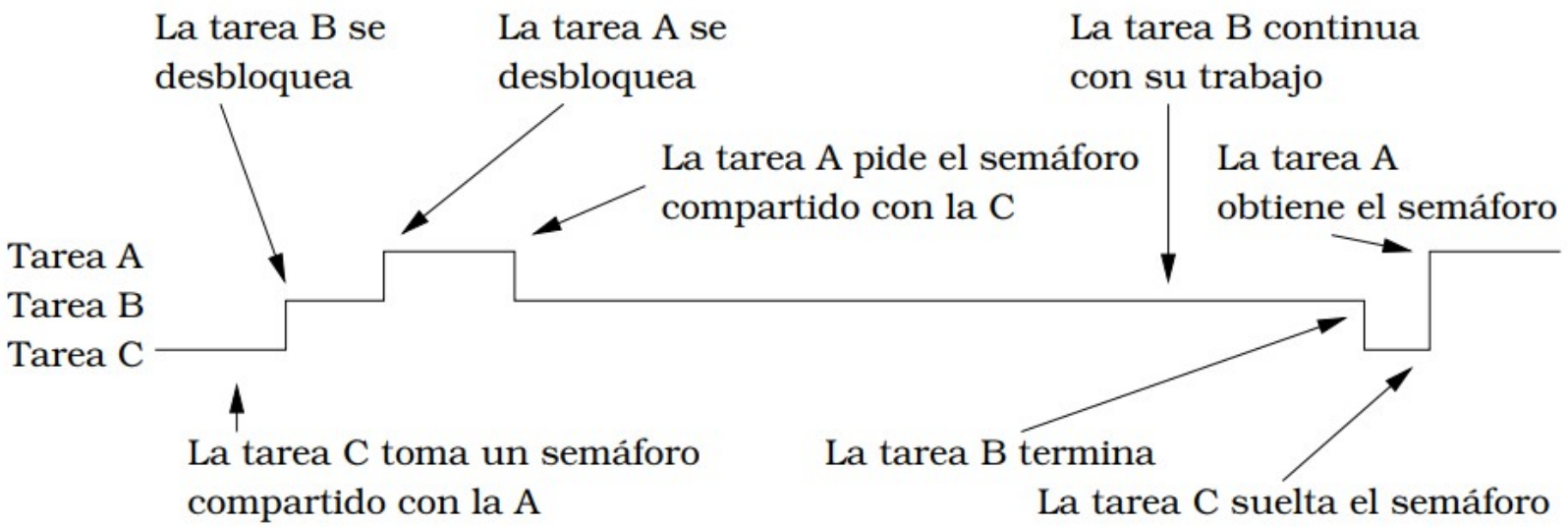
Sincronización y comunicación

- Semáforos:
  - **Sincronización de Tareas:** También pueden usarse para sincronizar dos tareas entre sí, o una tarea con una rutina de atención a interrupción (ISR). Por ejemplo, una ISR puede liberar un semáforo para desbloquear una tarea que lo estaba esperando. Para liberar un semáforo desde una ISR en FreeRTOS, se usa xSemaphoreGiveFromISR().



Sincronización y comunicación

- Semáforos:
  - **Inversión de Prioridad:** El uso de semáforos puede causar inversión de prioridad, donde una tarea de baja prioridad impide la ejecución de una de alta prioridad. Los mutexes son semáforos que implementan herencia de prioridad para mitigar este problema.

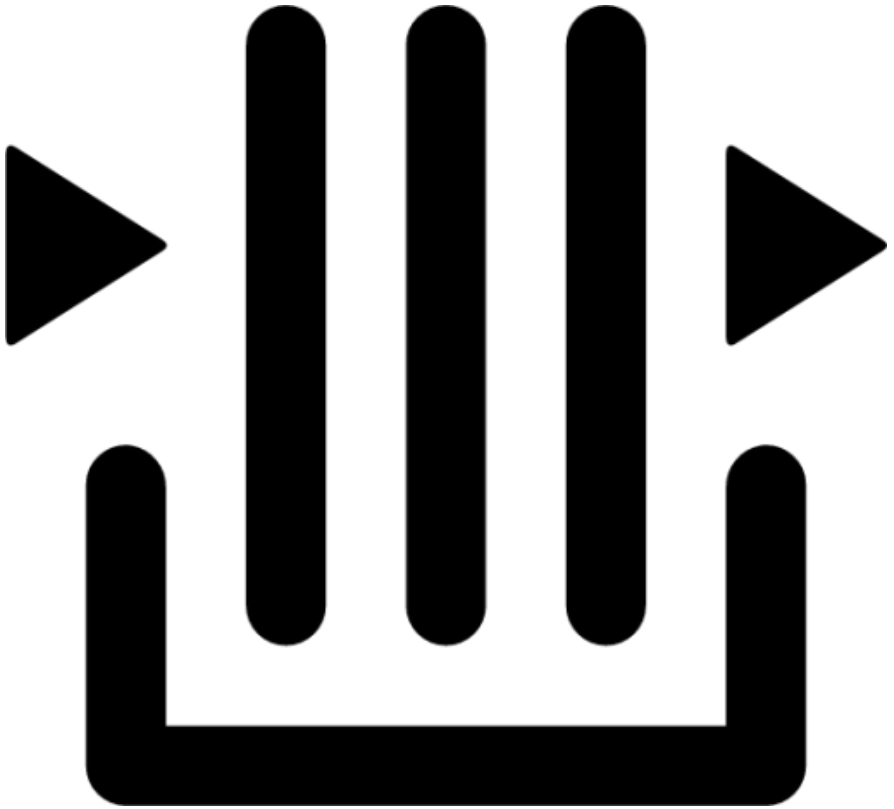


Sincronización y comunicación

- Semáforos:
  - **Abrazo Mortal (Deadlock):** Otro problema es el "abrazo mortal", donde dos o más tareas se bloquean mutuamente esperando semáforos que solo la otra puede liberar. Esto se puede mitigar con el uso de timeouts al intentar adquirir semáforos.

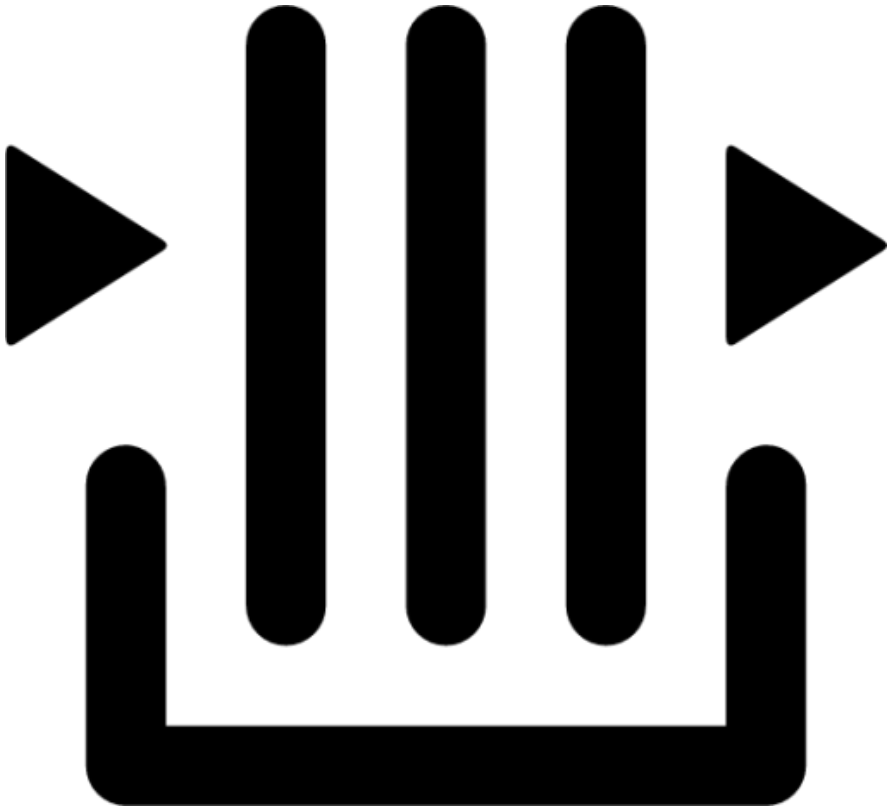
Sincronización y comunicación

- Colas:
  - **Comunicación Asíncrona:** Permiten la comunicación entre tareas y sirven como almacenamiento temporal para ráfagas de datos. Son útiles cuando hay múltiples generadores de datos y un solo consumidor.



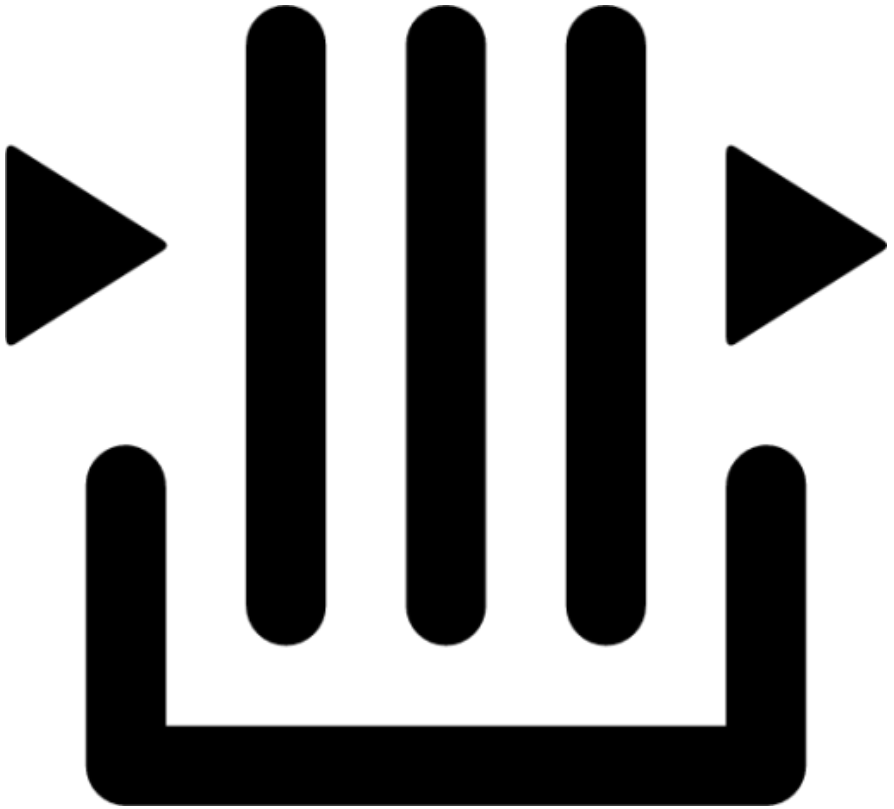
Sincronización y comunicación

- Colas:
  - **Creación y Operación:** Las colas se crean mediante llamadas al sistema (ej., xQueueCreate en FreeRTOS), que asigna la memoria y estructuras de control. El envío (xQueueSend) y recepción (xQueueReceive) de datos también se hace exclusivamente a través de llamadas al sistema.



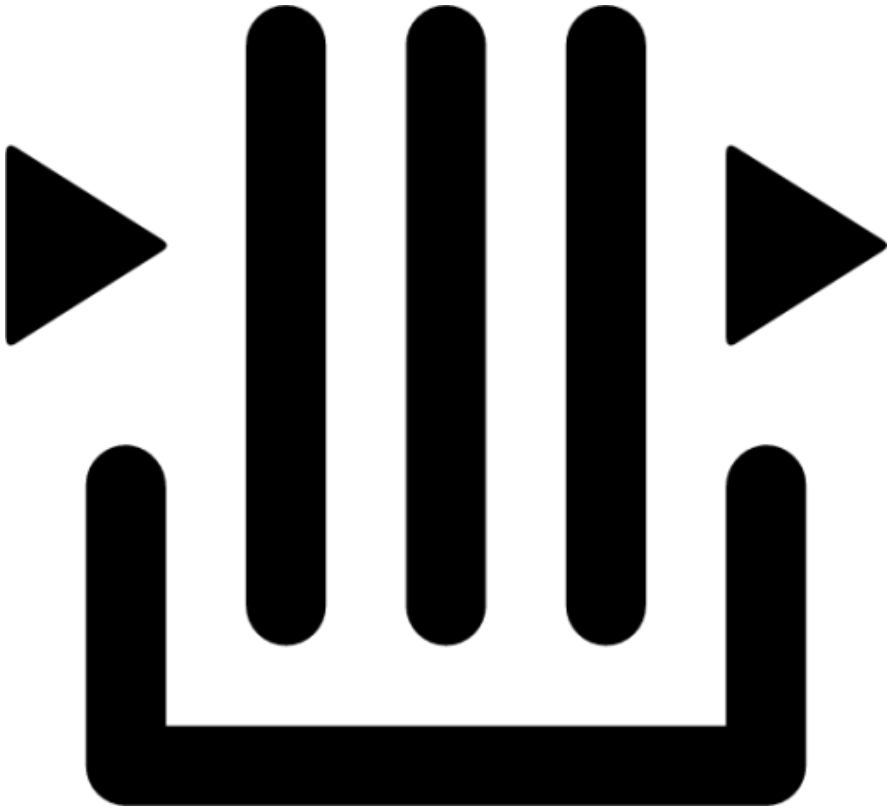
Sincronización y comunicación

- Colas:
  - **Bloqueo:** Las funciones de envío/recepción pueden bloquear la tarea si la cola está llena o vacía, respectivamente, lo cual es una ventaja sobre los sistemas Foreground/Background..



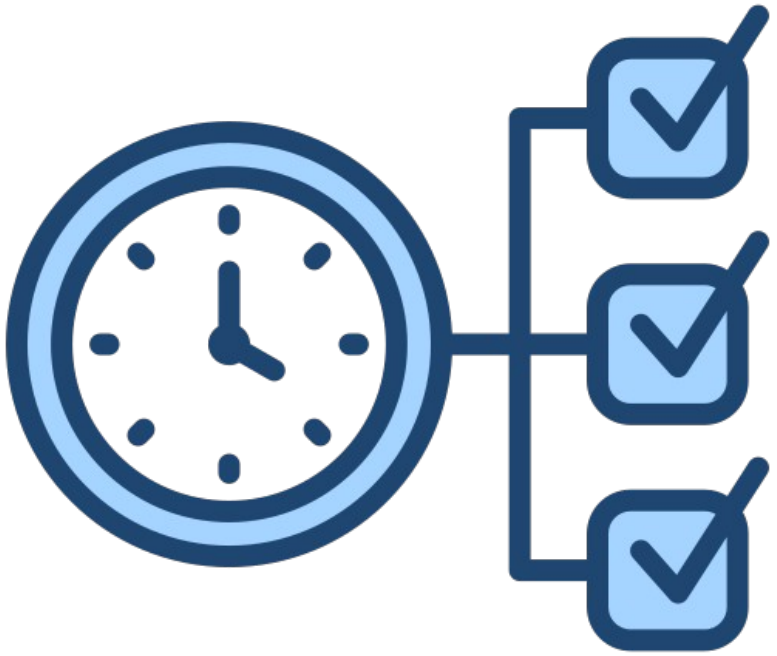
Sincronización y comunicación

- Colas:
  - **Desde ISRs:** Para enviar datos a una cola desde una ISR en FreeRTOS, se usa xQueueSendFromISR. Para recibir, se usa xQueueReceiveFromISR. Estas funciones no bloquean la ISR. Si una de estas llamadas despierta una tarea, es necesario forzar un cambio de contexto al finalizar la ISR con taskYIELD().



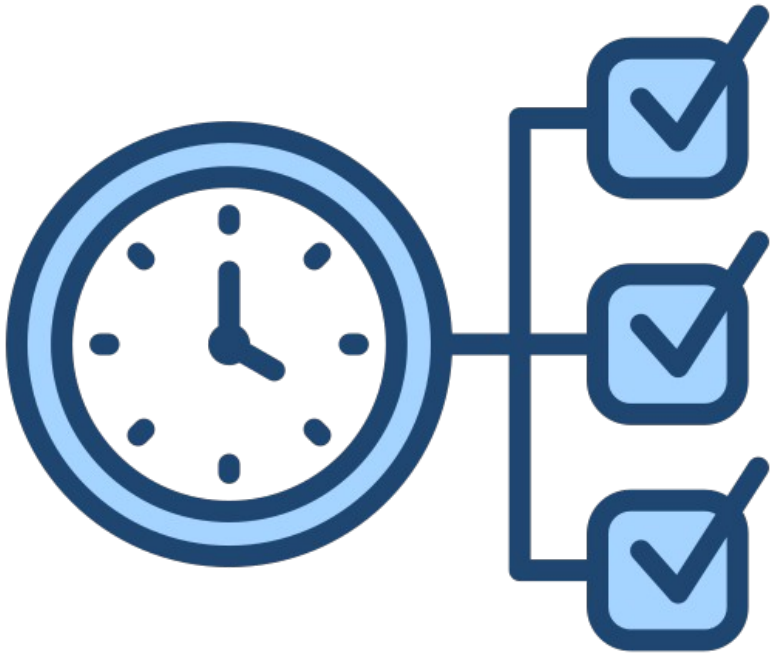
Gestión de tiempos en tareas

- Los SOTR facilitan la gestión del tiempo para las tareas, basándose en un contador que se incrementa periódicamente mediante una interrupción de temporizador (tick de reloj)



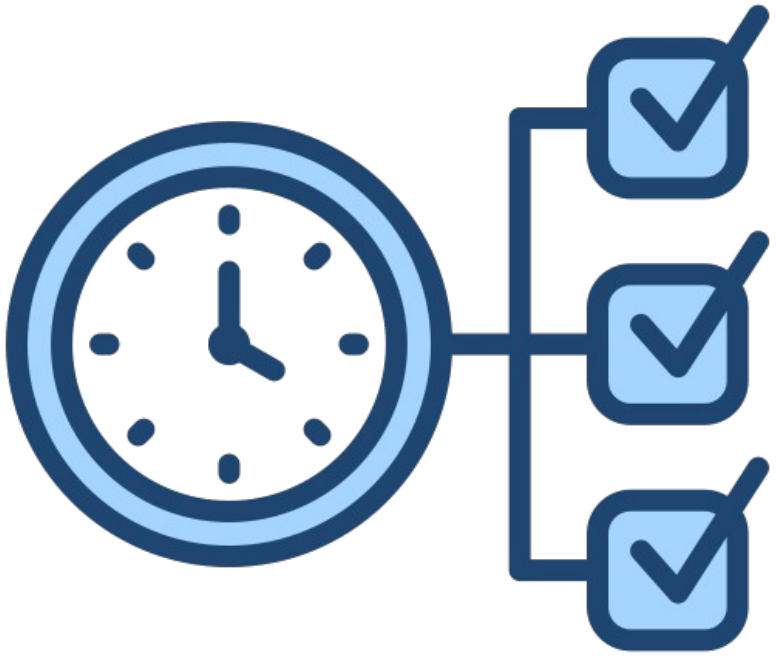
Gestión de tiempos en tareas

- Retardos:
  - vTaskDelay(xTicksToDelay): Suspende la tarea durante un número especificado de ticks. El retardo real puede variar ligeramente dependiendo del momento exacto de la llamada en relación con el tick de reloj.



Gestión de tiempos en tareas

- Tareas Periódicas:
  - vTaskDelayUntil(pxPreviousWakeTime, xTimeIncrement): Es la función ideal para ejecutar una tarea periódicamente, ya que despierta la tarea en un instante determinado, asegurando un periodo constante. El retardo real puede ser mayor si hay tareas de mayor prioridad listas para ejecutarse.



Resumen

- En resumen, las tareas en FreeRTOS son funciones C con bucles infinitos que son el corazón de la aplicación. Se gestionan por un planificador basado en prioridades, permitiendo la multitarea. La comunicación y sincronización entre ellas, así como la gestión de tiempos, se realiza a través de mecanismos específicos del RTOS como semáforos y colas, lo que garantiza el determinismo y la respuesta en tiempo real requeridos por las aplicaciones embebidas.